EXECUTABLE MARKDOWN FOR AI WORKFLOWS

Let AI handle ambiguity. Keep control in code.

Natural language, exact logic, and explicit guards.
One readable, runnable document.

Markdown-native

CLI + VS Code notebooks

INVOICE.MD

ONE DOCUMENT. SHARED VARIABLES.

```spl
let document = read_file('invoice.txt')
```

```
> Extract line items from @document.
> @validate[Invoice] save to @invoice
```

```spl
assert abs(
  invoice.subtotal + invoice.tax
  - invoice.total
) < 0.01 'totals do not add up'
success invoice
```

Condensed illustration. Invoice schema omitted.

THE GAP BETWEEN A PROMPT AND A PROCESS

A clever prompt isn't a reliable workflow.

A model can interpret the messy parts. The surrounding process still needs exact rules.

01

Instructions are not checks.

"Return valid data" is a request. A schema and an arithmetic assertion are executable tests.

02

Reasoning is not permission.

Drafting an action and executing it should have visible, separately controlled boundaries.

03

A good answer is not a regression test.

Live model output varies. Teams need a way to revisit recorded answers while changing code and guards.

The design opportunity: keep model judgment flexible without making control flow, validation, and I/O equally probabilistic.

ONE FILE. TWO WAYS TO EXPRESS INTENT.

A document people can read. A workflow the runtime can execute.

SPL fences run Python-like code. Blockquotes run natural-language agent commands.

01 / EXACT INPUT

Read, fetch, compute

Deterministic control flow and direct tools. No model needed for ordinary code.

→

02 / MODEL JUDGMENT

Interpret the messy part

Generate with `ask` or `think`; act with `agent` or a blockquote.

→

03 / EXPLICIT CONTROL

Validate, then proceed

Check structure and business rules. Limit actions. Pause for a person when needed.

Shared variables, functions, and active restrictions connect the layers.

Natural language is executed as an agent command; it is not compiled into SPL code.

Sources: README / Language / Natural language & models

FROM DOCUMENT UNDERSTANDING TO CHECKED DATA

AI reads the invoice. Code checks the math.

1. **Extract into a contract**
   A JSON Schema defines required fields, types, and line items.

2. **Check business truth**
   Recompute line totals and verify subtotal + tax = total.

3. **Write within a boundary**
   The output write is scoped to the invoice directory.

Valid JSON alone does not mean the numbers are correct.

ADAPTED FROM EXTRACT-INVOICE.MD

SCHEMA / INPUT SETUP OMITTED

````
> Extract the invoice in @document.
> @validate[Invoice] @retries[2] save to @invoice

```spl
let computed = 0
for line in invoice.lines {
    computed += line.quantity * line.unit_price
}
assert abs(computed - invoice.subtotal) < 0.01
assert abs(invoice.subtotal + invoice.tax
    - invoice.total) < 0.01
restrict './out/invoices' {
    write_file('./out/invoices/{invoice.number}.json',
        json_stringify(invoice, 2))
}
success invoice
```
````

Source: lang/examples/extract-invoice.md / Condensed excerpt, not the complete runnable file

KEEP GUIDANCE AND GUARANTEES SEPARATE

Make the boundary visible.

Use model instructions for intent. Use runtime mechanisms for enforceable checks.

`fmt / schema / assert`

Validate the result

Strict output parsing, JSON Schema, and exact business-rule checks before downstream work.

`ask / think / agent`

Separate generation from action

`ask` and `think` have no tools. Agent-enabled work can act within active restrictions.

`restrict fs / net / exec / mcp`

Scope tool access

Bound folders, hosts, executables, and MCP tools. Nested restrictions can only narrow access.

`human / @HUMAN / checkpoint`

Reserve decisions for people

Pause, publish the question, and resume with the answer without repeating completed calls.

Honest boundaries: restrictions are opt-in, not an OS sandbox. Prompt guidance is not validation. Failures do not roll back earlier actions.

Sources: Guards & failures / CLI & runtime / Specification

AUTOMATE PREPARATION. PRESERVE APPROVAL.

Let the model propose. Let a person authorize.

The issue-triage example combines exact GitHub CLI calls, classification, checks, and a human gate.

01 / FETCH

Read issues

`gh issue list` fetches an exact batch of unlabeled issues.

02 / CLASSIFY

Build a plan

The model selects a label, priority, and reason for each issue.

03 / VERIFY

Check coverage

Code rejects missing, duplicate, or out-of-batch decisions.

04 / APPROVE

Ask a human

The run checkpoints and waits before applying the plan.

05 / APPLY

Label issues

Exact `gh issue edit` calls run only after approval.

Model responsibility

Interpret the issue body and propose a classification.

Runtime + human responsibility

Validate the plan, authorize the change, execute exact commands.

Source: lang/examples/triage-issues.md / Requires authenticated gh and a checkpoint journal

A REUSABLE PATTERN, NOT A SINGLE-PURPOSE AGENT

Start wherever judgment meets a checkable outcome.

DEVELOPER WORKFLOWS

Review & release

Review source changes, structure findings, or turn git history into release notes.

Concrete check
Fail CI when high-severity findings remain.

`code-review.md` `release-notes.md`

DATA & DOCUMENTS

Extract & transform

Read invoices or translate documentation while retaining structural requirements.

Concrete check
Reconcile totals; preserve links, code, and headings.

`extract-invoice.md` `translate-docs.md`

OPERATIONS & RESEARCH

Explain & synthesize

Turn logs into incident narratives or collected web material into a sourced digest.

Concrete check
Compute log statistics exactly; verify citations.

`incident-report.md` `web-digest.md`

These are repository examples, not customer deployments or measured business outcomes.

Source: docs-template/Examples.md and linked runnable workflows

A SMALL SURFACE AREA FOR THE FIRST WORKFLOW

Write Markdown.
Run from the terminal or notebook.

invoice.md / SPL workflow notebook — Illustrative view, not a screenshot

SPL CELL / EXACT INPUT

```
let document = read_file('invoice.txt')
```

NATURAL-LANGUAGE CELL

```
Extract @document into @invoice.
@validate[Invoice]
```

SPL CELL / EXACT CHECK

```
assert invoice.total >= 0
invoice.total
```

Shared state across executed cells

CLI for repeatable runs

Arguments, machine-readable results, exit codes, model selection, and MCP configuration.

Editor support while you write

Parser diagnostics, completions, hover, definitions, snippets, and SPL highlighting.

Interactive, still Markdown

Run notebook cells with shared state. Save the original workflow format, not a separate notebook file.

Node.js runtime; model calls use GitHub Copilot. Notebook execution requires a trusted workspace.

Sources: lang/README.md / vscode/README.md / Notebook illustration omits setup and schema

ITERATE ON LOGIC WITHOUT ANOTHER LIVE MODEL CALL

Record the answer. Revisit the checks.

CAPTURE

Run with a trace

Save model requests, raw answers, and failures to a local JSONL trace.

→

REFINE

Edit code and guards

Change downstream logic while preserving matching model requests.

→

REPLAY

Test recorded answers

Rerun against the trace without contacting Copilot or MCP servers.

```
$ spl flow.md --trace run.jsonl
$ spl flow.md --replay run.jsonl --silent
```

Replay is not a dry run: explicit file, shell, and web calls still execute. Agent actions are not recreated. Changed model requests fail exact matching. Keep traces private.

Source: CLI & runtime / Traces and replay

THE MISSING MIDDLE BETWEEN SCRIPTS AND PROMPTS

Code when you know the steps. Language when you need judgment.

EXACT CODE

Specify the procedure

Control flow, calculations, parsing, and explicit I/O belong in executable logic.

Precise **Checkable**

SPL BRINGS THEM TOGETHER

One readable execution path.

Shared bindings connect intent to exact code. Guards make the handoff explicit.

Executable Markdown

NATURAL LANGUAGE

Describe the intent

Interpretation, classification, extraction, and synthesis benefit from model judgment.

Expressive **Flexible**

A focused workflow language, not full Python, a general workflow cache, or a transactional execution platform.

START NARROW. EARN A REPEATABLE WORKFLOW.

Proposed strategy / Not traction

Start with developers who already automate in text.

An initial adoption path to validate with users, not a claim about existing customers.

01

One useful local task

Try issue triage, a review report, or invoice extraction using a repository example.

02

A team-owned workflow

Keep the Markdown in version control. Review the prompts, checks, and permitted actions together.

03

Embed where it fits

Use the exported runtime API when a host application needs workflow execution or interactive sessions.

Measure in a pilot: Time to first useful run Manual correction time Validation failure rate Repeat usage

Audience and adoption sequence are hypotheses / Embedding API is documented in lang/README.md

A WORKING FOUNDATION. A FOCUSED VALIDATION PLAN.

The building blocks are here. The next proof is real usage.

Present in the repository

Interpreter + CLI + embedding API	Runtime v1.4.0*
Schemas, assertions, restrictions	Documented
Trace/replay + human checkpoints	CLI features
VS Code language support + notebooks	Source + tests
Workflow examples + offline test suites	Included

Proposed next milestones

1. Run a bounded pilot on a repeated, human-reviewed task.
2. Record representative answers and test the important failure paths.
3. Measure correction effort and repeat use before expanding scope.

No revenue, adoption, fundraising, or benchmark claims are inferred from the source tree.

*Version from lang/package.json / Presence in source is not production certification

FROM A USEFUL PROMPT TO A CONTROLLED PROCESS

Bring one real workflow.

Make the judgment explicit.
Make the checks executable.
Keep the whole thing readable.

github.com/dgawlik/semantic-prompt-language
Documentation & runnable examples ↗

01 **Pick a bounded task**

Choose recurring work with a concrete output and a human fallback.

02 **Define the contract**

Specify the schema, assertions, permissions, and approval points.

03 **Compare against today's process**

Measure correction time, failures caught, and whether the team uses it again.

INSPECTABLE CLAIMS

Built from the project, not invented metrics.

Project README

Product definition, install flow, and core value proposition.

Examples catalog + linked workflows

Invoice extraction, issue triage, review, documents, and operations.

Guards & failures

Restrictions, assertions, failure semantics, and human checkpoints.

CLI & runtime

Trace/replay, MCP, authentication, and embedding.

VS Code README

Language-server features, notebook workflow, and execution boundaries.

Specification / **Runtime manifest**

Goals, non-goals, generation contracts, and repository version.

How to read this deck

Capabilities describe the local repository snapshot reviewed in October 2026. Linked HEAD pages may change later.

Code on slides is condensed and labeled; full examples include setup, schemas, and error messages. The notebook view is illustrative.

Adoption strategy, pilot metrics, and next milestones are proposals, not measured traction or a committed roadmap.

Model calls depend on GitHub Copilot. Live output is nondeterministic. Restrictions are not an OS sandbox, and replay is not a side-effect-free dry run.

No market size, customer logos, financial projections, competitive benchmarks, or fundraising terms were supplied or fabricated.